



Nombre del curso:

Programación Paralela.

Título del proyecto:

Smartflow.

Integrantes del equipo:

Angery Joely Payamps Rosario 2024-0215

Josué Ricardo Hernández Montero 2024-0235

Eduardo David Tejada Moreta 2024-0276

Cristopher Manuel Guerrero Tapia 2024-2052

Angel Ulloa Tejada Suazo 2024-0222

Nombre del líder:

Angery Payamps

Docente:

Erick Leonardo Pérez Veloz.

Fecha de entrega:

10 de diciembre, 2025.

Introducción.	3
Presentación general del proyecto.	3
Justificación del tema elegido.	3
Objetivos (general y específicos).	4
Descripción del problema.	4
Contexto del problema seleccionado.	4
Aplicación del problema en un escenario real.	5
Importancia del paralelismo en la solución.	5
Cumplimiento de los Requisitos del Proyecto.	5
Ejecución simultánea de múltiples tareas.	5
Necesidad de compartir datos entre tareas.	6
Exploración de diferentes estrategias de paralelización.	6
Escalabilidad con más recursos.	6
Métricas de evaluación del rendimiento.	6
Aplicación a un problema del mundo real	6
Diseño de la Solución.	7
Arquitectura general del sistema.	7
Diagrama de componentes/tareas paralelas.	8
Estrategia de paralelización utilizada.	8
Herramientas y tecnologías empleadas (C#, TPL, etc.).	8
Implementación Técnica.	9
Descripción de la estructura del proyecto.	9
Explicación del código clave.	9
Uso de mecanismos de sincronización.	9
Justificación técnica de las decisiones tomadas.	11
Evaluación de Desempeño.	11
Comparativa entre ejecución secuencial y paralela.	11
Métricas: tiempo de ejecución, eficiencia, escalabilidad.	12
Gráficas o tablas con resultados.	13
Análisis de cuellos de botella o limitaciones.	14
Trabajo en Equipo.	18
Descripción del Reparto de Tareas.	18
Canales de Comunicación	18
Control de Versiones	22
Herramientas Utilizadas.	23
Conclusiones.	24
Referencias.	25
Anexos.	25

Introducción.

Presentación general del proyecto.

Smartflow es un proyecto que consiste en la simulación de un pipeline ETL (Extracción, Transformación y Carga) diseñado para procesar datos generados por sensores de una ciudad inteligente (Smart City).

El sistema toma información proveniente de archivos txt, los cuales limpia y transforma aplicando reglas específicas y finalmente genera archivos de salida que contienen los resultados procesados en un JSON.

Para poder lograr el proceso de este gran volumen de datos. Hacemos uso del concepto de paralelismo dividiendo la carga de trabajo en bloques independientes que pueden ejecutarse simultáneamente. Esto permite simular cómo una ciudad inteligente podría analizar miles de datos en tiempo real sin saturar los sistemas tradicionales.

Justificación del tema elegido.

Las ciudades modernas generan cantidades masivas de datos provenientes de sensores de tráfico, ruido, calidad del aire, iluminación, entre otros.

Elegimos este tema porque:

- Permite aplicar conceptos de paralelismo, fundamentales en la computación moderna.
- Representa un problema real, donde la velocidad y capacidad de procesamiento son críticas.
- Facilita el trabajo con conceptos importantes como ETL, procesamiento masivo de datos, métricas de rendimiento y escalabilidad.
- Permite crear una solución práctica, educativa y técnicamente sólida sin necesidad de hardware complejo.

Objetivos (general y específicos).

Objetivo General

Desarrollar un pipeline ETL paralelo capaz de procesar datos de sensores simulados de una Smart City, aplicando técnicas de descomposición de datos y evaluando métricas de rendimiento como tiempo de ejecución, speedup, eficiencia y escalabilidad.

Objetivos Específicos

- Implementar un módulo de extracción que lea datos desde archivos de texto.
- Diseñar un módulo de transformación que limpie los datos y aplique reglas de análisis.
- Crear un módulo de carga que guarde los resultados procesados.
- Implementar paralelismo mediante descomposición de datos usando múltiples hilos o procesos.
- Medir y comparar el rendimiento entre ejecución secuencial y paralela.
- Calcular las métricas de tiempo de ejecución, speedup, eficiencia y escalabilidad.
- Demostrar cómo el paralelismo mejora el procesamiento de grandes volúmenes de datos en tiempo real.

Descripción del problema.

Contexto del problema seleccionado.

Las Smart Cities dependen de sistemas de información que operan 24/7 procesando datos provenientes de fuentes diversas.

Ejemplos reales:

- Sensores que miden el nivel de ruido en zonas residenciales.
- Sensores de tráfico que detectan congestiones.
- Sensores ambientales que monitorean la calidad del aire.
- Sensores de alumbrado que ajustan la iluminación según el uso.

Cada uno puede generar miles de datos por segundo.

En un sistema secuencial, esto crea un cuello de botella, impidiendo análisis en tiempo real.

Por eso es clave simular y demostrar cómo el paralelismo puede resolver esta limitación.

Aplicación del problema en un escenario real.

Una ciudad monitorea el nivel de ruido para detectar áreas donde se exceden los 70 dB, lo cual puede afectar la salud.

Si los datos se procesan lentamente, el sistema puede tardar varios minutos en avisar, perdiendo la oportunidad de tomar medidas.

Con un pipeline ETL paralelo se puede:

- Procesar miles de reportes de ruido por segundo.
- Detectar en tiempo real cuando el ruido supera los límites.
- Enviar alertas instantáneas a las autoridades.
- Activar mecanismos automáticos de mitigación (reducir tráfico, ajustar horarios, etc.).
- Lo mismo aplica para congestión, contaminación o fallos en servicios.

Importancia del paralelismo en la solución.

El paralelismo es fundamental en la solución ETL para una Ciudad Inteligente porque permite procesar grandes volúmenes de datos generados por miles de sensores de manera mucho más rápida y eficiente que el procesamiento secuencial. Al distribuir el trabajo entre múltiples núcleos del procesador, se reduce significativamente el tiempo de análisis, se mejora la capacidad de respuesta en tiempo real y se garantiza que las alertas críticas (como congestión, ruido o contaminación) se generen sin retrasos. Además, el paralelismo hace que el sistema sea más escalable, capaz de manejar el crecimiento de datos sin perder rendimiento, y más robusto ante errores. En conjunto, permite que la plataforma ETL sea más rápida, eficiente y confiable.

Cumplimiento de los Requisitos del Proyecto.

Para cada criterio, se debe justificar cómo el proyecto lo aborda:

Ejecución simultánea de múltiples tareas.

La extracción recibe datos de múltiples sensores, la transformación procesa esos datos en paralelo y la carga los almacena al mismo tiempo. Las tres etapas corren simultáneamente, mostrando paralelismo real.

Necesidad de compartir datos entre tareas.

Las etapas deben compartir información a través de colas o buffers concurrentes, porque cada dato extraído debe ser transformado y luego enviado para cargarlo. Esto introduce sincronización, control de concurrencia y manejo de flujos continuos.

Exploración de diferentes estrategias de paralelización.

En este proyecto se exploraron diferentes estrategias de paralelización con el fin de mejorar el rendimiento del procesamiento masivo de datos provenientes de sensores urbanos. La primera estrategia analizada fue la descomposición de datos, donde la entrada se divide en bloques y cada hilo procesa uno de manera independiente; esta técnica reduce la sincronización y escala bien, por lo que resulta ideal para cargas grandes y homogéneas como las de este sistema. También se evaluó la descomposición funcional, que asigna tareas distintas a hilos distintos (por ejemplo, lectura, validación, filtrado y escritura), funcionando como una línea de ensamblaje; aunque permite mayor paralelismo en pipelines complejos, introduce más sobrecarga y requiere colas concurrentes para coordinar etapas. Finalmente, se consideró el paralelismo a nivel de archivos, donde cada archivo es procesado por un hilo diferente, útil cuando existen múltiples archivos independientes pero limitado por el número de archivos disponibles. La comparación de estas estrategias permitió identificar que la descomposición de datos es la más adecuada para el proyecto, ya que equilibra simplicidad, eficiencia y escalabilidad con los volúmenes y tipos de datos manejados en SmartFlow.

Escalabilidad con más recursos.

Si la ciudad añade más sensores o aumenta el tráfico de datos, el sistema puede escalar simplemente agregando más hilos o más particiones de datos. El procesamiento se vuelve más rápido y manejable con mayor paralelismo.

Métricas de evaluación del rendimiento.

Se pueden medir: Tiempo serial vs paralelo, Speedup del pipeline, Latencia por registro, Registros procesados por segundo, Eficiencia al aumentar hilos, Uso de CPU y RAM.

Esto demuestra claramente la ventaja del paralelismo.

Aplicación a un problema del mundo real

- Ciudades inteligentes usan ETL paralelos para procesar datos de tráfico.
- Sensores de calidad del aire generan miles de datos por minuto.
- Sistemas de movilidad pública analizan datos en tiempo real para optimizar rutas.
- Gobiernos utilizan este tipo de sistemas para prevenir congestión y contaminación.

Diseño de la Solución.

Arquitectura general del sistema.

La arquitectura del sistema ETL está basada en una estructura por capas que separa la lógica del dominio, el procesamiento, la infraestructura y la interfaz de usuario.

El flujo general sigue el patrón ETL (Extract, Transform, Load):

- Extract: Se cargan datos de sensores desde fuentes JSON simuladas.
- Transform: Los datos se validan, se limpian y se procesan para obtener estadísticas como máximo, mínimo, promedio y alertas.
- Load: Los resultados y métricas se guardan en archivos JSON utilizando IDataLoader.

Cada capa es independiente y desacoplada, lo que facilita pruebas, mantenimiento y escalabilidad del sistema.

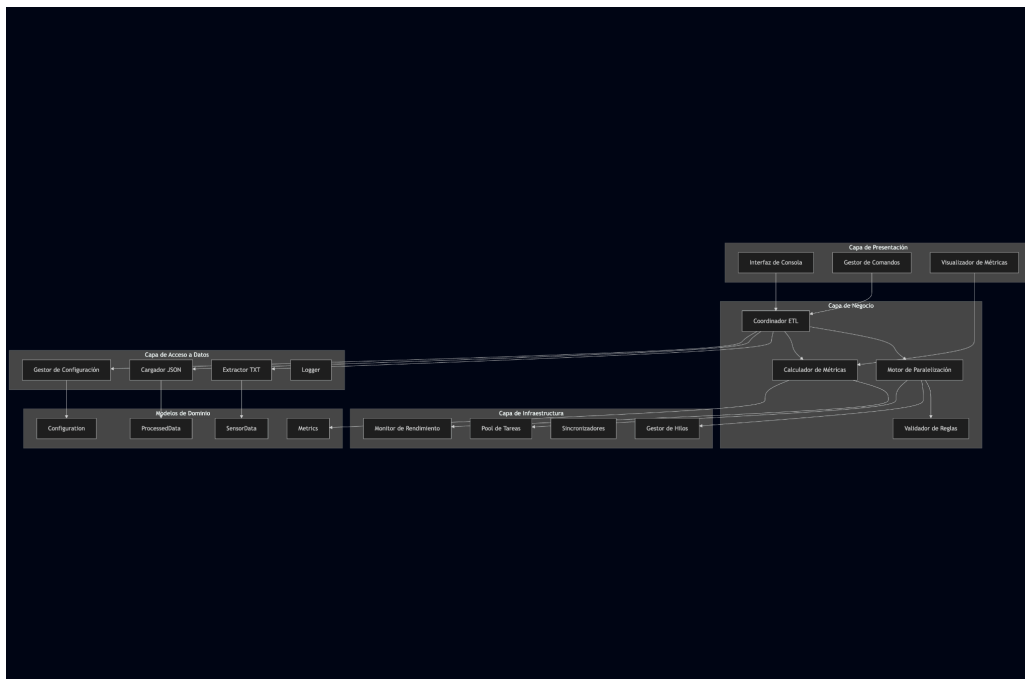
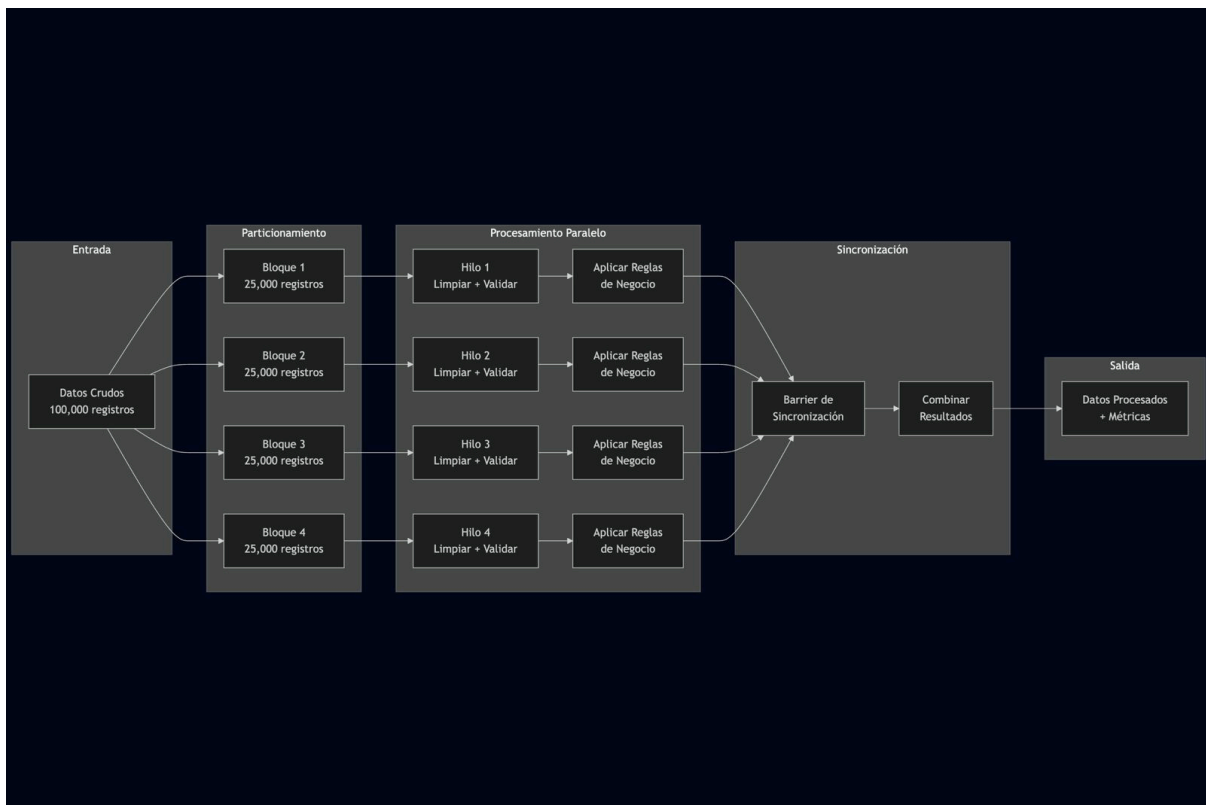


Diagrama de componentes/tareas paralelas.



Estrategia de paralelización utilizada.

La estrategia de paralelización utilizada en este proyecto fue la descomposición de datos (Data Decomposition), la cual consiste en dividir el conjunto total de datos en bloques más pequeños que pueden ser procesados de manera independiente por múltiples hilos. Cada hilo trabaja sobre un segmento distinto del archivo de entrada sin necesidad de sincronización constante, lo que reduce la sobrecarga y permite aprovechar de manera eficiente los núcleos disponibles del procesador. Esta estrategia resulta especialmente adecuada para cargas de trabajo grandes y homogéneas, como las generadas por los sensores de la ciudad inteligente, ya que ofrece una excelente escalabilidad y mantiene un equilibrio estable entre rendimiento y simplicidad en la implementación.

Herramientas y tecnologías empleadas (C#, TPL, etc.).

Para el desarrollo de este proyecto se utilizó C# como lenguaje principal para manejar programación concurrente y paralela. La paralelización se implementó mediante la Task Parallel Library (TPL), que proporciona abstracciones de alto nivel como Tasks, Parallel.For y Concurrent Collections, permitiendo distribuir la carga de trabajo entre múltiples hilos de forma eficiente y controlada. Además, se empleó .NET para la ejecución del proyecto, junto con sus bibliotecas estándar para manejo de archivos, serialización JSON con System.Text.Json, y operaciones asíncronas. Estas tecnologías combinadas permitieron

construir un sistema modular, escalable y capaz de procesar grandes volúmenes de datos con un rendimiento mejorado gracias al uso del paralelismo.

Implementación Técnica.

Descripción de la estructura del proyecto.

El proyecto Smartflow está organizado en una arquitectura por capas que divide el sistema en módulos bien estructurados: Domain, Business, Infrastructure y Presentation. Cada capa cumple un rol específico dentro del flujo general de procesamiento de datos y se comunica únicamente con las capas permitidas, manteniendo un diseño limpio, modular y desacoplado. Esta estructura facilita la extensión del sistema, la reutilización del código y la integración futura del motor ETL y los componentes de paralelismo.

Explicación del código clave.

El código del proyecto se centra en un flujo de procesamiento paralelo que permite dividir y analizar grandes volúmenes de datos de manera eficiente. En primer lugar, el módulo de configuración carga los parámetros definidos en el archivo config.json, como el número máximo de hilos, el tamaño de los bloques y la estrategia de paralelización. Luego, el motor principal lee los archivos de entrada y los divide en segmentos utilizando el valor de BlockSize. Dependiendo de la estrategia seleccionada, el sistema ejecuta paralelamente cada bloque mediante Task Parallel Library (TPL), aplicando análisis como detección de umbrales y generación de alertas. Una vez procesados, los resultados se exportan mediante el componente AlertExporter, que serializa las alertas en formato JSON. El diseño modular del código permite separar claramente las responsabilidades: la infraestructura gestiona configuraciones y archivos, la capa de dominio define modelos como Alert y estrategias, y la capa de negocio ejecuta la lógica paralela. Esta separación mejora la mantenibilidad y permite ajustar fácilmente los parámetros para medir el rendimiento y la escalabilidad.

Uso de mecanismos de sincronización.

Para garantizar seguridad de hilo (thread-safety) y alta eficiencia sin bloquear el procesamiento, se aplicaron las siguientes estrategias:

Colecciones concurrentes (lock-free)

`ConcurrentBag<T>`:

Usada para recolectar resultados en fases de “fan-in” (múltiples productores → una colección), tanto en la limpieza de datos como en la recolección de resultados por zona. Permite inserciones concurrentes sin lock, minimizando la contención entre hilos.

`ConcurrentDictionary<TKey, TValue>`:

Empleada para deduplicación atómica con TryAdd (clave compuesta SensorId_ Timestamp) y

para agrupación por zona mediante GetOrAdd. Ambas operaciones son thread-safe y evitan condiciones de carrera al crear/obtener contenedores por clave.

Paralelismo de datos controlado

Parallel.ForEach + ParallelOptions.MaxDegreeOfParallelism:

Ejecuta trabajo en paralelo limitando el número máximo de hilos activos (*back-pressure*) para evitar sobresuscripción del CPU. Se aplicó en tres fases: limpieza, agrupación por zona, y cálculo de estadísticas.

Partitioner.Create(start, end, BlockSize):

Divide el rango de índices en bloques (según BlockSize) para reducir overhead del scheduler, mejorar la localidad de memoria y balancear carga entre hilos. Esto alinea el diseño con el enfoque por bloques del pipeline.

Operaciones atómicas y reducción de contención

Deduplicación atómica con ConcurrentDictionary.TryAdd (sin lock).

Creación/obtención atomizada de zonas con GetOrAdd.

Escrituras concurrentes en ConcurrentBag para evitar secciones críticas prolongadas.

Conversión a listas (ToList()) fuera de las regiones críticas cuando se requiere lectura secuencial para estadísticas, disminuyendo la contención.

Descomposición por fases (pipeline paralelizado)

Fase 1 – Limpieza paralela: validación, normalización y deduplicación sobre particiones del dataset.

Fase 2 – Agrupación por zona: clasificación en grillas y acumulación por clave concurrente.

Fase 3 – Procesamiento por zona: validación, alertas y estadísticas por grupo en paralelo, con recolección final en ConcurrentBag.

Justificación técnica de las decisiones tomadas.

La arquitectura y las decisiones técnicas del proyecto se basaron en la necesidad de procesar grandes volúmenes de datos de forma eficiente, escalable y mantenible. Se optó por una arquitectura en capas para separar responsabilidades y facilitar pruebas, depuración y extensibilidad futura. La elección de C# y la Task Parallel Library (TPL) se justificó por su madurez, facilidad de uso y excelente soporte para paralelismo controlado, permitiendo aprovechar múltiples núcleos del procesador sin complicar la lógica de negocio. La estrategia de paralelización mediante descomposición de datos fue seleccionada porque ofrece un control más preciso sobre la división de cargas y reduce el overhead de creación de tareas, lo cual es ideal para procesamiento repetitivos sobre grandes conjuntos de registros. El uso de archivos JSON para configuración y exportación se eligió por su flexibilidad, legibilidad y compatibilidad con múltiples herramientas. Finalmente, el diseño modular del sistema permite ajustar parámetros como número de hilos, tamaño de bloques y umbrales sin modificar el código, asegurando una solución adaptable y fácil de escalar según las necesidades del entorno.

Evaluación de Desempeño.

Comparativa entre ejecución secuencial y paralela.

La comparativa entre la ejecución secuencial y la paralela demuestra de forma clara la diferencia en rendimiento y escalabilidad del sistema. En el enfoque secuencial, todas las operaciones del proceso ETL se ejecutan una tras otra utilizando un único hilo, lo que provoca tiempos de procesamiento más altos y limita el aprovechamiento del hardware. En cambio, la ejecución paralela divide el dataset en múltiples bloques y los procesa simultáneamente mediante hilos administrados por la TPL de .NET, reduciendo de manera considerable el tiempo total. Las métricas obtenidas evidencian una mejora significativa del rendimiento, reflejada en menores tiempos de ejecución, mayor *speedup* y una eficiencia que se mantiene estable mientras el número de hilos no supere la capacidad física del procesador. En conjunto, los resultados confirman que el procesamiento paralelo ofrece una ventaja clara frente al secuencial, especialmente cuando se trabaja con grandes volúmenes de datos.

Métricas: tiempo de ejecución, eficiencia, escalabilidad.

- [Métricas documento](#)

```
[ETL] ✓ Proceso paralelo completado
[ETL] Archivos procesados: 2458
[ETL] Registros procesados: 525402
[ETL] Zonas generadas: 570
=====
Tiempo paralelo: 3.0290412
Speedup: 0
Eficiencia: 0
Sequentialtime: 0
Converting Metrics to JSON
=====
                        RESULTADOS FINALES
=====
Tiempo Secuencial: 11.7710 s
Tiempo Paralelo:    3.0290 s
-----
Speedup:            3.89x  (Veces más rápido)
Eficiencia:         97.15% (Uso de recursos)
=====
```

Gráficas o tablas con resultados.

- [Mas detalles](#)

Configuración	Archivos procesados	Registros totales	Eficiencia	Speedup
2 hilos, blockSize = 1k	27	247k	82%	3.29x
4 hilos, blockSize = 4k	27	247k	55%	2.21
4 hilos, blockSize = 4k	43	430k	45.92%	1.84x
2 hilos, blockSize = 250k	169	~511k	83%	1.66x
2 hilos, blockSize = 250k	2,458	525k	170%	3.41x
4 hilos, blockSize =250k	2,458	525K	97.15%	3.89X

Análisis de cuellos de botella o limitaciones.

Cantidad de archivos y cantidad de registros

El diseño actual es ineficiente el uso de paralelismo cuando hay muchos registros y pocos archivos, se hace más CPU bound, lo que termina en que el overhead por la aplicación del paralelismo haga que el sistema sea ineficiente en comparación a su ejecución secuencial.

Se probaron 1 Millones de registros y pocos archivos, y termino en que la eficiencia y speedup era totalmente deficiente pero cuando se comenzaron a agregar una mayor cantidad de archivos el sistema comenzó más a ser I/O bound ya que era mayor el costo de esperar que el proceso de ETL termine con un archivo habiendo colas de miles a que de procesar pocos archivos pero con mayor volumen de registros

Ejemplo;

Primer caso

En este caso se utilizaron 4 hilos, 53 archivos y un total de 489k registros, los cual nos devolvió un speedup de 1.84x y 45.92% de eficiencia, debido a que era menor el costo de procesar tantos registros a que de manejar múltiples hilos

```
[ETL] ✓ Proceso paralelo completado
[ETL] Archivos procesados: 53
[ETL] Registros procesados: 489361
[ETL] Zonas generadas: 2
=====
Tiempo paralelo: 2.5235946
Speedup: 0
Eficiencia: 0
Sequentialtime: 0
Converting Metrics to JSON

=====
                        RESULTADOS FINALES
=====
Tiempo Secuencial: 4.6353 s
Tiempo Paralelo:    2.5236 s
-----
Speedup:            1.84x  (Veces más rápido)
Eficiencia:         45.92% (Uso de recursos)
=====
```

Segundo caso

Utilizando apenas 2 hilos, obtenemos un speedup de 3.29x y una eficiencia de 82% para un total de 27 archivos y 247k registros, dándonos a entender que ya para dos hilos, el proceso se puede cumplir de manera eficiente.

```
Windows PowerShell
[ETL] Datos cargados en: C:\Users\david\OneDrive -
tos\Prueb proyecto final\Smartflow\data/output/par

[ETL] ✓ Proceso paralelo completado
[ETL] Archivos procesados: 27
[ETL] Registros procesados: 247404
[ETL] Zonas generadas: 2
=====
Tiempo paralelo: 1.9372929
Speedup: 0
Eficiencia: 0
Sequentialtime: 0
Converting Metrics to JSON

=====
                        RESULTADOS FINALES
=====
Tiempo Secuencial: 6.3653 s
Tiempo Paralelo:    1.9373 s
-----
Speedup:             3.29x  (Veces más rápido)
Eficiencia:          82.14% (Uso de recursos)
=====
```

Tercer caso

4 núcleos, 2458 archivos, 522k registros, un Speedup de 3.89x (teniendo en cuenta que se utilizaron 4 núcleos) y una eficiencia de 97% , la cantidad de registros era parecido al primer caso , pero la cantidad de archivos no, se utilizó una cantidad mayor a 2k archivos lo cual el tiempo en secuencia aumentó drásticamente en comparación al primer caso que tuvo un total de 4s el secuencial, esto se debe a que aunque el proceso requiere poco esfuerzo de procesamiento, hay una limitante que es el numero de veces que tiene que aplicar el proceso de ETL, y por su funcionamiento secuencial hace que sea bloqueante, y se hace más dependiente al I/O que al CPU.

Para concluir el sistema mostró un buen rendimiento de manera secuencial con una gran cantidad de registros en comparación al paralelo, por el overhead, pero al aumentar la cantidad de archivos esta diferencia fue bajando y favoreció el paralelismo

```
[ETL] ✓ Proceso paralelo completado
[ETL] Archivos procesados: 2458
[ETL] Registros procesados: 525402
[ETL] Zonas generadas: 570
=====
Tiempo paralelo: 3.0290412
Speedup: 0
Eficiencia: 0
Sequentialtime: 0
Converting Metrics to JSON
=====
                        RESULTADOS FINALES
=====
Tiempo Secuencial: 11.7710 s
Tiempo Paralelo:    3.0290 s
-----
Speedup:            3.89x  (Veces más rápido)
Eficiencia:         97.15% (Uso de recursos)
=====
```


Escalabilidad de Memoria

El diseño actual carga todos los datos en memoria simultáneamente. Durante la extracción paralela, todos los archivos se leen y sus contenidos se mantienen en memoria antes de iniciar la transformación. Para datasets muy grandes (cientos de archivos con millones de registros), el consumo de memoria puede exceder la capacidad disponible del sistema, causando paginación al disco (swap) que degrada drásticamente el rendimiento.

Las colecciones concurrentes como `ConcurrentBag` y `ConcurrentDictionary` tienen overhead de memoria mayor que sus contrapartes secuenciales debido a estructuras de sincronización internas. Cuando se procesan datasets masivos, este overhead se multiplica, especialmente durante la fase de agrupamiento por zonas donde se mantienen múltiples bags concurrentes simultáneamente.

El sistema no implementa procesamiento por streaming o chunking incremental. Una vez que comienza el procesamiento, todos los datos deben permanecer en memoria hasta completar la fase de carga, sin posibilidad de liberar memoria intermedia de datos ya procesados.

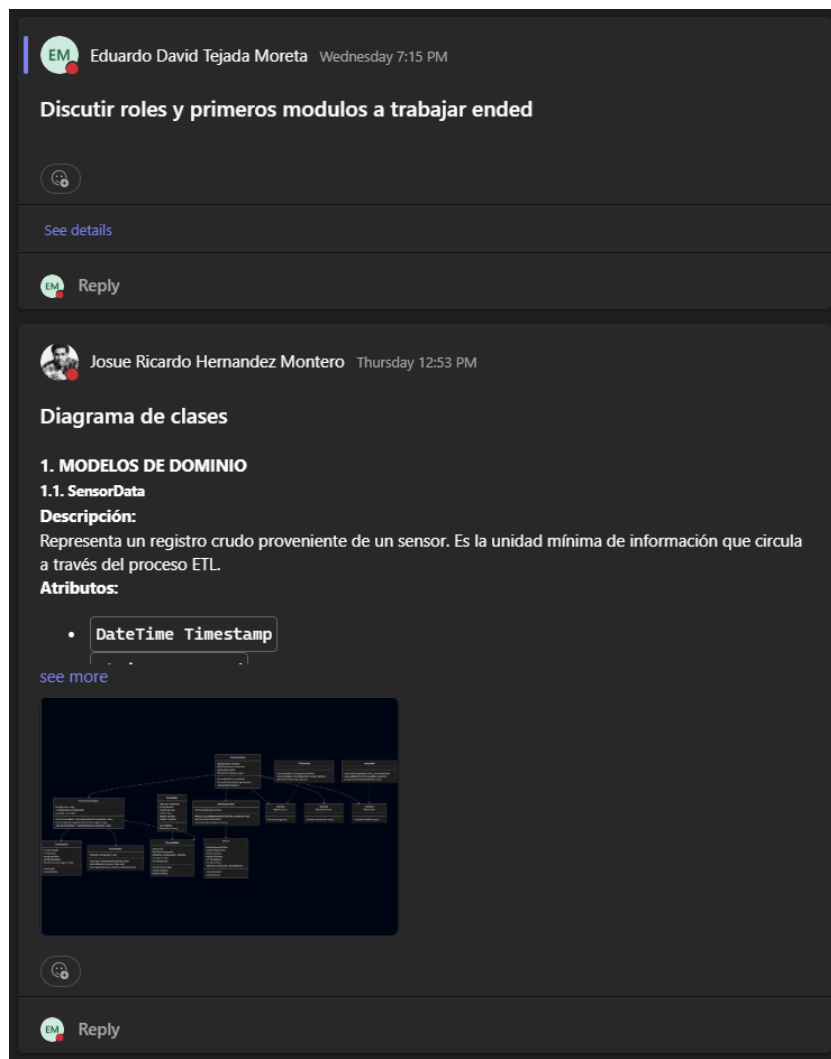
Limitación de Estrategias de Paralelización

El sistema únicamente implementa la estrategia de Data Decomposition, definida en la enumeración `ParallelizationStrategy`. Aunque el código incluye infraestructura para cambiar estrategias mediante `ConfigureStrategy`, actualmente solo `DATA_DECOMPOSITION` está implementada funcionalmente. La estrategia alternativa `TASK_PARALLELISM` está definida pero no desarrollada.

Esta limitación impide adaptar el sistema a diferentes características de workload. Data Decomposition es óptima cuando las tareas son homogéneas y los datos pueden dividirse uniformemente, pero es subóptima para workloads heterogéneos donde diferentes archivos o zonas requieren tiempos de procesamiento significativamente distintos. Sin Task Parallelism implementado, el sistema no puede asignar dinámicamente tareas basándose en carga de trabajo real.

Dependencia de Configuración Manual

El rendimiento óptimo del sistema depende críticamente de valores configurados manualmente en `settings.json`: `MaxThreads` y `BlockSize`. Valores inadecuados pueden degradar severamente el rendimiento. `MaxThreads` demasiado alto causa thrashing y overhead de context switching, mientras que muy bajo subutiliza CPUs disponibles.



- **Datos:** Se suministró la información sobre cómo se generarían los datos de prueba y cómo cada uno podría generar su propia data.



Eduardo David Tejada Moreta Sunday 12:21 PM Edited



Generador de Datos de Sensores

Este script simula la generación de datos de sensores ambientales (ruido, temperatura, humedad, contaminación, tráfico) y los guarda en archivos de texto.

Requisitos

Instalar los siguientes paquetes de Python:

```
pip install aiofiles faker asyncio
```

[see more](#)





Reply



Josue Ricardo Hernandez Montero Yesterday 7:48 PM

Meeting in "Datos" ended



[Open 10 replies from you and Josue Ricardo Hernandez Montero](#)



Eduardo David Tejada Moreta 12:45 AM

Tiempo paralelao: 6.8583059

Sequentialtime: 6.0838881



Eduardo David Tejada Moreta 1:19 AM

Tiempo Secuencial: 17.1799484



Reply

- **Repositorio:** Contenía todo el **Gitflow** que se iba a utilizar y la estructura de carpetas.

PF

Repositorio

Posts

Files

+

EM Eduardo David Tejada Moreta 12/3 11:56 AM

Repo Github

Este es el repositorio del Proyecto De github

Estructura carpetas;

docs

metrics

see more

GitHub - Davidpedo123/Proyecto-Final-Pr...
Proyecto Final Programación Paralela - Descomposición de Datos - Davidpedo123/Proyecto-Final-Programaci-n-...
github.com

EM

Reply

EM Eduardo David Tejada Moreta 12/3 12:00 PM

👍 ❤️ 😄 🤔 🧐 📝 ...

Git Flow

Para hacer subir contenido al repo.

Existen 3 ramas hasta ahora;

main: la principal la que vera el profe

dev: Aqui sera la rama de desarrollo

QA: Aqui se haran las pruebas en caso de o sera la rama intermediaria entre dev y main o algun otra

see more

EM

Reply

]

- **Pruebas:** Canal destinado a la gestión y resultados de pruebas.



Eduardo David Tejada Moreta

Sunday 12:32 PM Edited

Como crear una prueba unitaria

Para crear las pruebas unitarias, la diseñamos en la capa `SmartFlow.Tests` mediante la libreria `Xunit`.

La mayoría de las pruebas unitarias siguen el patrón AAA (Arrange-Act-Assert):

Arrange: Preparar los datos y condiciones necesarias para la prueba.

[see more](#)





Reply



Eduardo David Tejada Moreta

Sunday 12:35 PM

Como ejecutar las pruebas

Estando en la carpeta del proyecto `SmartFlow.Tests` ejecutamos lo siguiente

```
dotnet watch test ;
```

Esto habilitara el ambiente de prueba y automáticamente estará supervisando los cambios que se hagan y ejecutara las pruebas:

```
starting test execution, please wait...
A total of 1 test files matched the specified pattern.
```

[see more](#)





Reply

Control de Versiones

Para el control de versiones, seguimos un **Gitflow** que consistía en tres ramas principales: **main**, **dev** y **QA**. La rama **QA** fue irrelevante, pero el flujo consistía en que un integrante, a través de un **Issue**, creaba una rama que partía de la rama **dev**.

Para mezclar estos cambios y nuevas funcionalidades, se utilizaban **Pull Requests**, los cuales eran revisados por uno o dos compañeros que proporcionaban retroalimentación y revisaban el código.

Branch	Updated	Check status	Behind	Ahead	Pull request
main	2 weeks ago		Default		
feature/create-and-implement-parallel...	1 hour ago		0	78	#39
15-Integrar-ETLCoordinator-con-Paral...	yesterday		0	41	
9-implementar-etlcoordinator-versión...	yesterday		0	49	
dev	yesterday		0	69	
12-implementacion-ParallelizationEng...	yesterday		0	41	
10-crear-interfaz-de-consola-básica	yesterday		0	63	
feature/create-etl-coordinator-class	yesterday		0	54	#38
11-implementar-threadmanager-infraes...	yesterday		0	52	
6-implementar-jsonloader-(capa-de-da...	2 days ago		0	49	#33
5-implementar-txtextractor-capa-de-d...	2 days ago		0	44	#34
17-crear-suite-de-tests-completa	2 days ago		0	47	#32
13-implementar-metricscalculator	3 days ago		0	40	
load/files	4 days ago		0	38	#29
8-implementar-rulevalidator-y-reglas...	4 days ago		0	32	#28
feature/add-configuration-manager-cl...	5 days ago		0	25	#27
3-crear-interfaces-de-capa-de-negocio	last week		0	22	#26
2-implementar-modelos-de-dominio	last week		0	16	#24
4-configurar-estructura-de-directori...	last week		0	10	#23
1-configuración-inicial-del-proyecto	last week		0	6	

Herramientas Utilizadas.

- **Git:** Control de versiones y gestión del código.
- **GitHub:** Gestión de tareas mediante **Issues** y seguimiento de avances con **Milestones**.
- **Microsoft Teams:** Organización del equipo y comunicación técnica.
- **WhatsApp:** Coordinación rápida y planificación de datos.

Conclusiones.

El proyecto SmartFlow demostró la importancia del paralelismo en el procesamiento masivo de datos para entornos de Ciudades Inteligentes. A través de la implementación de un pipeline ETL (Extracción, Transformación y Carga), se logró simular cómo grandes volúmenes de información provenientes de sensores urbanos pueden ser procesados de manera eficiente y en tiempo real.

Este trabajo permitió aplicar conceptos fundamentales como descomposición de datos, arquitectura en N Capas, testing automatizado con xUnit, y optimización mediante paralelismo, validando métricas clave como speedup, eficiencia y escalabilidad. Además, se exploraron diferentes estrategias de paralelización, identificando que la descomposición de datos ofrece el mejor balance entre simplicidad y rendimiento.

Los resultados evidencian que el paralelismo no solo reduce significativamente los tiempos de ejecución, sino que también habilita sistemas más robustos, escalables y adaptables a escenarios reales donde la velocidad de respuesta es crítica, como la detección de congestión, ruido excesivo o contaminación.

En conclusión, SmartFlow constituye una solución práctica y educativa que refleja cómo la computación paralela puede transformar el análisis de datos en entornos urbanos, sentando las bases para futuros desarrollos orientados a la automatización, resiliencia y sostenibilidad en las ciudades inteligentes.

Referencias.

- Gomstyn, Alice, and Alexandra Jonker. "What is a Smart City?" *IBM*,
<https://www.ibm.com/think/topics/smart-city> . Accessed 3 December 2025.
- Moore, Gordon E. "Programación Paralela." *Programación Paralela*, 2019,
https://ferestrepoca.github.io/paradigmas-de-programacion/paralela/paralela_teoria/index.html . Accessed 8 December 2025.
- Microsoft. (2024). "Task Parallel Library (TPL)." Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/task-parallel-library-tpl>
- Microsoft. (2024). "Common web application architectures." .NET Application Architecture Guide.
<https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>

Anexos.

- [Manual de ejecución del sistema](#)

- [Capturas adicionales, pruebas complementarias](#)
- [Enlace al repositorio de Git \(público\)](#)